



Infineon Summer School Functional Verification

Anul
2026



Cuprins

Introducere	5
Conectare la server.....	5
Mediul de lucru și simulatorul	7
Coding style și debugging	7
Referințe teoretice.....	7
Descrierea templateului	9
Ziua 1	11
Discuții:	11
Sarcini de lucru:	11
Ziua 2	13
Discuții	13
Sarcini de lucru:	13
Ziua 3	13
Discuții:	13
Sarcini de lucru:	14
Ziua 4	14
Discuții:	14
Sarcini de lucru:	15
Ziua 5	15
Discuții	15
Sarcini de lucru:	16
Ziua 6	16
Discuții:	16
Sarcini de lucru:	16
Ziua 7 -> 9.....	17
Discuții:	17



Introducere

Conectare la server

Acest document conține informațiile necesare pentru implementarea corectă și completă a aplicației de verificare funcțională din cadrul Infineon Summer School ediția 2025.

Pe toată durata desfășurării cursurilor se va folosi mediul de lucru disponibil la adresa **sonic2.dcae.pub.ro** sau **sonic3.dcae.pub.ro** în funcție de unde vi s-a spus ca este arondat userul primit. Pentru conectarea la server se va folosi o conexiune SSH. Pentru o experiență mai plăcută și intuitivă de lucru se recomandă instalarea [VsCode](#) împreună cu extensiile [SystemVerilog - Language Support](#), [EditorConfig](#), [Better ToDo Tree](#) și [Remote - SSH](#) disponibile în cadrul magazinului de extensii integrate în aplicație.

Pentru conectarea în mod grafic la server se poate folosi una din variantele de mai jos:

1. **RECOMANDAT:** Folosirea unei conexiuni VNC:
 - a. Descărcați clientul VNC Tiger VNC și deschideți-l.
 - b. În câmpul adresă introduceți adresa serverului unde este alocat numele vostru de utilizator
 - c. Apăsăți butonul de conectare/connect după care selectați utilizatorul vostru și introduceți parola aferentă acestuia.
 - d. Alegeți sesiunea sau apăsați nu session dacă lista este goală
 - e. Acum ar trebui să vă aflați în interfața Desktop

Atunci când doriți să încheiați sesiunea de lucru grafic, **folosiți funcția de LOGOFF** pentru a preîntâmpina situația în care utilizatorul se blochează și devine inaccesibil.

Modul de lucru cu conexiune **VNC** ar trebui folosit **exclusiv pentru lucru cu forme de undă** și ar trebui închis în momentul în care nu mai este nevoie de formele de undă. Pentru **editarea fișierelor și compilare** folosiți conexiunea **ssh și modul de simulare batch**.

2. Folosirea Xforward împreună cu un Xserver instalat pe computerul local

Dacă utilizați un sistem de operare Microsoft Windows este necesară descărcarea aplicației [Windows X Server](#) din cadrul căreia veți porni programul XLaunch înainte de a porni conexiunea SSH.



Display settings

Select display settings
Choose how VcXsrv display programs

☒ Multiple windows


☐ Fullscreen


☐ One large window


☐ One window without titlebar


Display number
(Specify -1 to let vcxsrv automatically choose one)

< Back

Next >

Cancel

Client startup

Select how to start clients

☒ Start no client
This will just start the xserver. You will be able to start local clients later.

☐ Start a program
This will start a local or remote program which will connect to the xserver. You will be able to start local clients later too. Remote programs are started using SSH.

☐ Open session via XDMCP
This will start a remote XDMCP session. Starting local clients later is limited. This option is not available with the "Multiple windows" mode.

< Back

Next >

Cancel

Extra settings

Extra settings

☒ Clipboard
Start the integrated clipboard manager

☒ Primary Selection
Also map the PRIMARY selection to the windows clipboard.

☒ Native opengl
Use the native windows opengl library (wgl). Make sure to export the LIBGL_ALWAYS_INDIRECT environment variable.

☐ Disable access control
Use this when you want vcxsrv to accept connections from all clients.

Additional parameters for VcXsrv

< Back

Next >

Cancel

Finish configuration

Configuration complete

Configuration is complete. Click Finish to start VcXsrv.

You may also save the configuration for later use.

Save configuration

< Back

Finish

Cancel

Configurarea Xlaunch

Odată deschis VsCode va fi nevoie sa stabilim conexiunea SSH cu serverul precum in imaginile de mai jos.

REMOTE EXPLOR...

Remotes (T) v ...

REOTES (TUNNELS/SSH)

SSH

+

ssh -x student30@sonic.dcae.pub.ro

Press 'Enter' to confirm your input or 'Escape' to cancel

Select SSH configuration file to update

C:\Users\matachedrago\ssh\config

Select the platform of the remote host

Linux

Windows

macOS

Enter password for

Press 'Enter' to conf

Configurare conexiune SSH



Pentru numele de utilizator și parolă folosiți datele ce v-au fost transmise individual fiecăruia dintre voi.

Mediul de lucru și simulatorul

Pe durata acestui proiect se va utiliza IDE-ul VsCode cu conexiune SSH așa cum s-a arătat în capitolul de mai sus sau în cazuri excepționale nedit pentru a menține o performanță cât mai bună a serverului.

Simulatorul ce va fi utilizat este simulatorul comercial **CADENCE XCELIUM** pentru care există un număr de 30 de licențe disponibile. Din acest motiv la încetarea sesiunii de lucru **ESTE NECESARĂ INCHIDEREA SIMULATORULUI** și utilizarea opțiunii de **LOG-OFF** atunci când folosiți o sesiune VNC.

Modul de pornire al simulatorului poate fi consultat în fișierul **READ.md** localizat în rădăcina directorului **ifx_summer_school_2026**

Coding style și debugging

Pentru o utilizare judicioasă a spațiului de lucru oferit de către fișierul în care scriem se recomandă setarea caracterului TAB la 4 pct și, dacă este disponibilă opțiunea, înlocuirea automată a caracterului TAB cu SPACE. De asemenea, pentru a ușura munca de debug se recomandă utilizarea de mesaje în punctele cheie sau ori de câte ori se consideră ca fiind necesar prin utilizarea următoarei sintaxe:

```
` uvm_info(get_name(), $sformatf(„pattern_ca_in_c”), UVM_LOW)  
E.X: ` uvm_info(get_name(), $sformatf(„A=%0d”, a), UVM_LOW)
```

Pentru a crește gradul de productivitate și ușurința de debug se recomandă respectarea următoarelor reguli:

- Numele de variabile, funcții și taskuri vor fi semnificative și vor descrie modul de utilizare al acestora (ex: `get_data_pkg`, `interrupt_cnt`, etc)
- Pentru a crește lizibilitatea se recomandă utilizarea snake case pentru scrierea numelor de variabile și funcții (ex. `get_data_pkg`)
- Fiecare instrucțiune aflată în interiorul unui bloc **begin end** va fi indentată astfel încât codul să se afle în interiorul acestuia
E.X:

```
if (interrupt_cnt == 2) begin  
    execute_interrupt_chk();  
    ->check_e_trigger;  
end
```

Referințe teoretice

Pentru a ajuta în procesul de înțelegere al sarcinilor de lucru și în general al procesului de verificare se sugerează consultarea următoarelor linkuri:



- <https://www.asic-world.com/systemverilog/index.html>
- <https://www.chipverify.com/tutorials/uvm>
- <https://verificationacademy.com/forums/all-topics>
- <https://cluelogic.com/2011/07/uvm-tutorial-for-candy-lovers-overview/>

De asemenea, se recomanda si consultarea fișierelor din directorul docs disponibil împreună cu mediul de verificare în care se vor găsi manualele de referință atât pentru UVM cât și pentru limbajul System Verilog.

Utilizarea LLM-uri (i.e. ChatGpt / Gemini / DeepSeek) este de asemenea o practică încurajată pentru lămuriri suplimentare însă este descurajată pentru rezolvarea cerințelor efective. Rețineți faptul ca orice model AI poate inventa explicații/ detalii legate de întrebarea voastră motiv pentru care în timpul laboratoarelor este mai bine să întrebați instructorul.



Descrierea templateului

Pentru o descriere a fișierelor și directoarelor din acest template consultați fișierul README.md disponibil în directorul rădăcină.



Ziua 1

Discuții:

- Prezentarea domeniului verificării funcționale
- Prezentarea unui proces complet de verificare funcțională
- Demonstrație a unui mediu complet de verificare
- Scurtă recapitulare concepte SystemVerilog (porturi, instanțiere, tipuri de atribuire)
- Recapitulare sumară concepte POO(exemple folosind SystemVerilog)
- Prezentare sumară a sistemului de versionare Git

Sarcini de lucru:

- În interiorul directorului *ifx_summer_school_2026* să se inițializeze un repository local și configurați repositoryul astfel încât să permită realizarea de commituri.
- Să se configureze/creeze fișierul *.gitignore* astfel încât directorul *simulation* și tot ceea ce include să nu fie versionat.
- Creați un commit care să includă toate fișierele. La mesajul de commit scrieți „Add initial project version”.
- Creați un branch nou pe care să îl numiți „Ziua-1” și mutați-vă pe acesta.
- În interiorul testbench-ului(top) să se instanțieze o interfață de tip *“ifx_dig_interface”* numită *dig_if* și să se realizeze conexiunile dintre semnalele disponibile în interfață și cele disponibile în top.
- Să se instanțieze DUT-ul și să se realizeze conexiunile acestuia cu semnalele disponibile la nivelul top-ului.
- Să se scrie un task care permite configurarea modului de generare al ceasului de sistem în funcție de perioadă și unitatea de timp. Să se folosească acest task pentru a genera un ceas cu frecvența de 40 MHz.
- În directorul de teste să se creeze un fișier nou numit *ifx_dig_hello_world.svh* și să se includă acest fișier în interiorul pachetului *“ifx_dig_test_pkg.sv”*
- În fișierul nou creat *“ifx_dig_hello_world.svh”* să se scrie o clasă ce extinde clasa *uvm_test* și care implementează următoarea funcționalitate:
 - afișează mesajul „Start of test”,
 - realizează un reset ce ține 2 pulsuri de ceas,
 - așteaptă 500 us,
 - afișează mesajul „Hello UVM World!”,
 - așteaptă 500 us și încheie simularea.

Pentru implementarea cerințelor, la *run_phase* se va adăuga următoarea linie: ***uvm_config_db#(virtual ifx_dig_interface)::get(this, "", "dig_if", dig_vif)*** și se va folosi direct interfața, nu uitați că *dig_vif* este o variabilă ce trebuie declarată. Pentru scrierea acestui test puteți să vă inspirați din structura celorlalte teste disponibile în directorul de teste.



După finalizarea tuturor sarcinilor de lucru rulați o simulare urmând următorii pași:

- a) Asigurați-vă că terminalul de comandă deschis se află în directorul *sv_sim*.
- b) Rulați comanda: **`./run.py -mode sim -test ifx_dig_hello_world -seed 1234`**
- c) Realizați probe pe toate semnalele disponibile la nivelul testbenchului și adăugați semnalele din interfețele disponibile pe forma de undă(waveform window) și semnalele din interfața DUT-ului.
- d) Rulați testul
- e) Observați textul din consola simulatorului și forma de undă. Discutați aceste observații.
- f) Realizați operațiunea merge a branchului Ziua-1



Ziua 2

Discuții

- Structura de pachete (includes și imports)
- Instanțierea claselor
- Arhitectura generală a unui mediu de verificare
- Fazele de rulare UVM
- Agentul și componentele sale – privire de ansamblu

Sarcini de lucru:

- i. Creați un nou branch in proiectul git numit Ziua-2
- ii. În fișierul „*ifx_dig_regblock_pkg.svh*” să se realizeze includerea tuturor fișierelor necesare pentru blocul de registre.
- iii. În fișierul „*ifx_dig_data_bus_uvc_pkg.sv*” să se includă componentele aferente UVC-ului.
- iv. În fișierul „*ifx_dig_pkg.sv*” să se realizeze importul pachetelor aferente UVC-urilor.
- v. În fișierul „*ifx_dig_pkg.sv*” să se realizeze includerea tuturor componentelor mediului de verificare.
- vi. În fișierul „*ifx_dig_registers.svh*” să se implementeze clasele aferente registrilor din DUT. Folosiți ca exemplu clasa de mai sus ce implementează registrul CTRL0.
- vii. În fișierul „*ifx_dig_regblock.svh*” să se instantieze și configureze regiștrii aferenți DUT-ului. Utilizați exemplul deja existent pentru registrul CTRL0.
- viii. În fișierul „*ifx_dig_regblock.svh*” să se implementeze funcția *get_reg_by_name()* care primește ca parametru numele registrului căutat și întoarce obiectul aferent registrului. În cazul în care registrul nu poate fi găsit, funcția va genera o eroare ce va conține un mesaj sugestiv.
- ix. Adăugați în cadrul tuturor fazelor de rulare din cadrul „*ifx_dig_testbase.svh*”, „*ifx_dig_env.svh*”, „*ifx_dig_scoreboard.svh*” mesaje care să permită urmărirea acestora.
Ex. `uvm_info(get_name(), „ENV build_phase done”, UVM_LOW)
- x. În fișierul „*ifx_dig_hello_world.svh*” modificați testul astfel încât să extindă „*ifx_dig_testbase*” și rulați testul. Observați noua ierarhie și log-ul de simulare.
- xi. Realizați operația de merge dintre branchul zilei curente si master/main

Ziua 3

Discuții:

- Agentul și componentele sale (aprofundare, folosind codul din *ifx_dig_data_bus_uvc* ca exemplu)



- Secvențe
- Teste

Sarcini de lucru:

- i. Creați un nou branch numit Ziua-3
- ii. În interiorul fișierului „*ifx_dig_testbase.svh*” să se implementeze taskul *drive_reset()* capabil să genereze intrare asincronă în reset și ieșirea sincronă din acesta. Durata resetului trebuie să fie configurabilă atât în timp cât și în ciclul de ceas. Lungimea minimă a unui reset trebuie să fie 3 pulsuri de ceas.
- iii. Identificați testele disponibile și modificați-le astfel încât să folosească metoda *drive_reset* creată la punctul anterior.
- iv. În interiorul fișierului „*ifx_dig_testbase.svh*” să se implementeze taskul *drive_input_pulses()* capabil de a genera un număr configurabil de pulsuri cu o anumită lungime pe o intrare selectată de utilizator.
- v. Rulați testul „*ifx_dig_counter_mode_test.svh*”. Care este rezultatul testului? Discutați comportamentul observat. Modificați testul astfel încât acesta să genereze un număr de 10 pulsuri cu o perioadă de 100ns. Rulați iarăși testul și observați modificările.
- vi. În directorul de teste să se creeze un fișier nou denumit „*ifx_dig_sfr_test.svh*” care să conțină o clasă cu același nume ce se extinde din *ifx_dig_testbase.svh* și implementează următorul scenariu:
 - a. realizează un reset inițial al DUT-ului;
 - b. scrie la fiecare registru valoarea patternului;
 - c. citește valoarea tuturor registrelor; dă reset; schimbă patternul;
 - d. reia scenariul de la pasul 2 până ce s-au folosit toate patternurile.Discutați între voi care ar fi patternurile necesare și argumentați de ce patternul propus ar fi necesar. Observați tranzațiile pe de bus utilizând atât consola de simulare cât și forma de undă.
- vii. În interiorul fișierului „*ifx_dig_data_bus_uvc_sequence_lib.svh*” să se scrie o secvență care să permită realizarea unui acces de scriere urmat imediat de citire de la o adresă specificată.
- viii. Modificați testul „*ifx_dig_sfr_test.svh*” astfel încât să folosească secvența implementată la punctul 6. Pentru simplitate puteți implementa un task separat pentru acest scenariu în cadrul testului.
- ix. Realizați operația de merge dintre branchul zilei curente și master/main

Ziua 4

Discuții:

- Secvențe
- Constrângeri și randomizare



Sarcini de lucru:

- i. Creați un nou branch numit Ziua-4
- ii. În interiorul fișierului „*ifx_dig_sequences.svh*”, la nivelul clasei „*ifx_pin_toggle_sequence*”, să se implementeze logica necesară astfel încât aceasta să fie echivalentă din punct de vedere logic cu funcția *drive_input_pulses()*. Pentru aceasta va fi nevoie să:
 - a. Adăugați câmpurile necesare pentru configurare secvenței
 - b. Implementați logica de generare a pulsurilor.
- iii. Implementați următoarele constrângeri la nivelul secvenței „*ifx_pin_toggle_sequence*”:
 - a. Constrângere care să nu permită generare indexului unui pin ce nu există în design.
 - b. Constrângere care să nu permită generarea unui puls cu o lungime nerezonabilă ($\geq 100\mu s$)
 - c. Constrângere care să nu permită generarea unui număr negativ de pulsuri
- iv. În fișierul „*ifx_dig_pkg.svh*” să se modifice sequencerul declarat astfel încât acesta să folosească tipul corect de item.
- v. În fișierul „*ifx_dig_env.svh*” să declare și să se creeze o instanță a sequencerului *ifx_dig_pin_sequencer* numită *pin_sequencer*.
- vi. În interiorul „*ifx_dig_testbase.svh*” să se completeze taskul *read_reg()* care să permită realizarea unui acces de citire folosind secvența *fix_dig_data_bus_uvc_read_sequence*.
- vii. Să se rescrie testul „*ifx_dig_sfr_test.svh*” astfel încât acesta să utilizeze funcțiile *read_reg()* și *write_reg_fields()*. Să se implementeze posibilitatea ca datele ce se vor scrie în registre să fie aleatoare.
- viii. Rescrieți testul *ifx_dig_counter_mode_test* pentru a utiliza funcțiile *read_reg* și *write_reg_fields*. De asemenea, modificați testul astfel încât să utilizeze secvența „*ifx_dig_pin_toggle_sequence*” pentru a genera stimulii pe intrarea DUT-ului.
- ix. Realizați operația de merge dintre branchul zilei curente și master/main

Ziua 5

Discuții

- Porturi de analiză
- Golden model
- Scoreboarding
- Execuția paralelă a taskurilor



Sarcini de lucru:

- i. Creați un nou branch numit Ziua-5
- ii. În fișierul „*ifx_dig_env.svh*” să se realizeze conectarea portului de analiză al monitorului agentului de date la cel disponibil în scoreboard. Rulați unul dintre testele funcționale și observați ce modificare apare în log-ul de pe consolă.
- iii. În fișierul „*ifx_dig_scoreboard.svh*” adăugați în faza de build un apel la *uvm_config_db* astfel încât variabila *dig_vif* să fie populată.
- iv. Tot în fișierul de la punctul iii să se scrie un cod ce să permită execuția în paralel a taskurilor *golden_model*, *do_checkers*, *collect_coverage*
- v. În fișierul „*ifx_dig_golden_model*” completați taskul „*update_reg_variables()*” astfel încât:
 1. Modelul de registre este oglindit permanent
 2. Comanda de clear este detectată și evenimentul de clear asociat
 3. Frecvența PWM-ului este decodată din valoarea registrului
 4. Factorul de umplere este determinat din valoarea registrului
- vi. Completați logica taskului „*model_expected_output_o()*” astfel încât aceasta să emuleze modul de generare al semnalului de ieșire, puls respectiv palier. Ajutați-vă de instrucțiunile prezente în comentarii.
- vii. Realizați operația de merge dintre branchul zilei curent și master/main

Ziua 6

Discuții:

- Checkere
- Acoperire funcțională

Sarcini de lucru:

- i. Creați un nou branch numit Ziua-6
- ii. În fișierul „*ifx_dig_interface.svh*” implementați aserții care să permită verificarea faptului că semnalele de ieșire au valoare de reset corectă atunci când se încheie resetul.
- iii. Completați funcția *check_read_data()* astfel încât aceasta să verifice valoarea citită din DUT cu cea existentă în modelul de registre. În cazul unui acces la o adresă la care nu se află niciun registru se va verifica ca s-a citit valoarea 0.
- iv. În cadrul fiecărui checker existent în fișier adăugați un mesaj care să indice faptul că acesta a pornit/repornit.
- v. Rulați iarăși unul dintre teste. Ce modificări observați? Cum rulează checkerele? Există vreo problemă? Dacă da, propuneți o soluție pentru aceasta.
- vi. În fișierul „*ifx_dig_coverage.svh*” scrieți un covergroup care să demonstreze că fiecare intrare a fost selectată pentru numărare în toate modurile de funcționare disponibile counterului.



- vii. Adăugați alte covergroupuri suplimentare la care vă gândiți.
- viii. Completați covergroupurile deja existente cu alte crossuri pe care le considerați necesare.
- ix. Adăugați alte checkere pe care le considerați ca fiind necesare.

Ex:

```
covergroup my_covergroup@(coverage_e);  
  data_val_label: coverpoint data { bins cifre= {[0:9]};  
                                   bins numere= {[10:99]}; }  
  
  addr_val_label: coverpoint addr { bins low_addr= {[0:25]};  
                                   bins high_adrr= {[26:99]}; }  
  
  addr_x_data: cross data_val_label, addr_val_label;  
  
endcovergroup
```

Ziua 7 -> 9

Discuții:

- Debug pe cod
- Clarificări adiționale
- Implementare teste/funcționalități adiționale